

VLLM专题（三十二）——与 HuggingFace 的集成

AI专题精讲

已于 2025-03-19 10:19:26 修改

阅读量123

收藏

点赞数

版权

分类专栏：

大模型专题系列

文章标签：

人工智能



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

本文档描述了 vLLM 如何与 HuggingFace 库集成。我们将逐步解释在运行 `vllm serve` 时，底层发生了什么。

假设我们想通过运行 `vllm serve Qwen/Qwen2-7B` 来提供流行的 QWen 模型。

模型参数是 `Qwen/Qwen2-7B`。vLLM 通过检查相应的配置文件 `config.json` 来确定该模型是否存在。请参阅此代码片段以了解实现细节。在此过程中：

- 如果模型参数对应于现有的本地路径，vLLM 将直接从该路径加载配置文件。
- 如果模型参数是由用户名和模型名称组成的 HuggingFace 模型 ID，vLLM 将首先尝试使用 HuggingFace 本地缓存中的配置文件，使用模型参数作为模型名称，`--revision` 参数作为修订版本。请参阅他们的网站以了解 HuggingFace 缓存的工作原理。
- 如果模型参数是 HuggingFace 模型 ID，但在缓存中未找到，vLLM 将从 HuggingFace 模型中心下载配置文件。请参阅此函数以了解实现细节。输入参数包括模型参数作为模型名称，`--revision` 参数作为修订版本，环境变量 `HF_TOKEN` 作为访问模型中心的 token。在我们的例子中，vLLM 将下载 `config.json` 文件。

确认模型存在后，vLLM 加载其配置文件并将其转换为字典。请参阅此代码片段以了解实现细节。

接下来，vLLM 检查配置字典中的 `model_type` 字段以生成要使用的配置对象。vLLM 直接支持一些 `model_type` 值；请参阅此处以获取列表。如果 `model_type` 不在列表中，vLLM 将使用 `AutoConfig.from_pretrained` 加载配置类，使用模型、`--revision` 和 `--trust_remote_code` 作为参数。请注意：

- HuggingFace 也有自己的逻辑来确定要使用的配置类。它将再次使用 `model_type` 字段在 transformers 库中搜索类名；请参阅此处以获取支持的模型列表。如果未找到 `model_type`，HuggingFace 将使用配置 JSON 文件中的 `auto_map` 字段来确定类名。具体来说，它是 `auto_map` 下的 `AutoConfig` 字段。请参阅 DeepSeek 以获取示例。
- `auto_map` 下的 `AutoConfig` 字段指向模型仓库中的模块路径。要创建配置类，HuggingFace 将导入模块并使用 `from_pretrained` 方法加载配置类。这通常会导致任意代码执行，因此仅在启用 `--trust_remote_code` 时执行。

随后，vLLM 对配置对象应用一些历史补丁。这些主要与 RoPE 配置有关；请参阅此处以了解实现细节。

最后，vLLM 可以到达我们要初始化的模型类。vLLM 使用配置对象中的 `architectures` 字段来确定要初始化的模型类，因为它在注册表中维护了从架构名称到模型类的映射。如果注册表中未找到架构名称，则意味着 vLLM 不支持此模型架构。对于 `Qwen/Qwen2-7B`，`architectures` 字段是 `["Qwen2ForCausalLM"]`，它对应于 vLLM 代码中的 `Qwen2ForCausalLM` 类。该类将根据各种配置初始化自身。

除此之外，vLLM 还依赖 HuggingFace 完成另外两件事。

1. Tokenizer（分词器）

vLLM 使用 HuggingFace 的分词器对输入文本进行分词。分词器通过

`AutoTokenizer.from_pretrained` 加载，其中模型参数作为模型名称，`--revision` 参数作为修订版本。也可以通过指定 `vllm serve` 命令中的 `--tokenizer` 参数来使用其他模型的分词器。其他相关参数包括 `--tokenizer-revision` 和 `--tokenizer-mode`。请查阅 HuggingFace 的文档以了解这些参数的含义。这部分逻辑可以在 `get_tokenizer` 函数中找到。在获取分词器后，vLLM 会将分词器的一些计算成本较高的属性缓存起来，具体逻辑在 `get_cached_tokenizer` 中实现。

2. Model Weight（模型权重）

vLLM 从 HuggingFace 模型中心下载模型权重，使用模型参数作为模型名称，`--revision` 参数作为修订版本。vLLM 提供了 `--load-format` 参数来控制从模型中心下载哪些文件。默认情况下，它会尝试以 `safetensors` 格式加载权重，如果 `safetensors` 格式不可用，则回退到 PyTorch 的 `bin` 格式。我们还可以传递 `--load-format dummy` 来跳过权重下载。

推荐使用 `safetensors` 格式，因为它在分布式推理中加载效率高，并且可以避免任意代码执行的风险。有关 `safetensors` 格式的更多信息，请参阅文档。这部分逻辑可以在这里找到。需要注意的是：

- 如果模型权重以 `safetensors` 格式存储，vLLM 会优先加载这种格式。
- 如果 `safetensors` 格式不可用，vLLM 会回退到 PyTorch 的 `bin` 格式。
- 使用 `--load-format dummy` 时，vLLM 会跳过权重下载，通常用于测试或调试。

3. 总结

至此，vLLM 与 HuggingFace 的集成已经完成。

总的来说，vLLM 从 HuggingFace 模型中心或本地目录读取配置文件 `config.json`、分词器和模型权重。它使用来自 vLLM、HuggingFace transformers 的配置类，或者从模型仓库中加载配置类。通过这种集成，vLLM 能够高效地加载和运行 HuggingFace 模型，同时提供了灵活的配置选项以支持不同的使用场景。



AI专题精讲

已关注

0

0

0

分享

专栏目录

已订阅